

聚类算法核心知识整理

读书笔记与算法解析

目录

01

聚类基础概念

深入解析簇的定义、核心特征，并对比欧式距离与余弦距离两种关键的相似性度量方法，为后续算法理解奠定理论基础。

02

K-means算法详解

系统梳理K-means算法的核心原理与标准迭代步骤，重点探讨K值选择、初始质心选取等关键参数，并分析算法的优势与局限性。

03

层次聚类（HC）详解

详细介绍凝聚式层次聚类的核心过程，解析单链接、全链接等簇间距离度量方式，并深入解读树状图（Dendrogram）的分析与应用。

04

算法对比与总结

结合理论与实践案例，横向对比K-means与层次聚类的算法特性、计算效率，总结两种算法在不同数据场景下的适用边界与优化方向。

簇（Cluster）的概念

簇是聚类分析的核心概念，指数据集中一组具有高度相似性的数据点集合，也常被称作“类”或“群组”。聚类分析的本质，即是在无序数据中发现这类自然形成的簇结构，揭示数据内在的分布规律。

01 / 核心定义

簇的核心逻辑可概括为“簇内紧凑，簇间分离”。同一簇内的数据点具有极高的相似度和极小的差异性，而不同簇的数据点则表现出显著的相异性。这一特征是判断聚类结果合理性、衡量聚类算法有效性的最核心依据。

02 / 关键特征

同质性：同一簇内的样本在特征空间中空间距离相近，且数据结构、属性分布与变化模式高度一致，呈现出紧密的聚合状态。

异质与无监督：簇间样本差异显著，且聚类无需预设标签，仅基于数据自身的相似性度量完成划分，是探索性分析的关键手段。

03 / 典型应用场景

在商业领域，可用于客户分群以实现精细化营销与服务；在计算机视觉中，用于图像分割提取核心区域；在风控与安全领域，用于异常检测识别欺诈交易或网络攻击；在生物医学中，可分析基因表达模式，揭示基因间的关联与功能聚类规律。

类度量方法：欧式距离 & 余弦距离

01 欧式距离 (Euclidean Distance)

计算两个样本在n维特征空间中的直线距离，本质上衡量的是样本间的**绝对差异**，反映了样本在数值大小上的直接差距，是最直观的几何距离度量方式。

$$d(x,y) = \sqrt{[\sum(x_i - y_i)^2]}$$

（其中 n 为特征维度， x_i 、 y_i 分别为两样本在第 i 维上的特征值）

适用场景：适用于数据量纲统一、特征数值的绝对差异具备实际业务意义的场景，例如客户消费金额分析、商品价格差异比较等。

02 余弦距离 (Cosine Distance)

通过计算两个样本向量夹角的余弦值来度量相似度，核心衡量的是样本间的**方向差异**，不受向量模长（数值绝对大小）的影响，更关注样本的趋势一致性。

$$\cos\theta = [\sum(x_i y_i)] / [\sqrt{\sum x_i^2} \cdot \sqrt{\sum y_i^2}]$$

（余弦值越接近 1，代表两向量方向越一致，相似度越高）

适用场景：广泛应用于文本相似度匹配、用户行为偏好分析、推荐系统等场景，例如判断两篇文章主题是否相似、用户兴趣是否契合。

K-means: 核心原理与算法步骤

01 核心原理：物以类聚，迭代优化

算法预先设定簇的数量 k ，通过不断迭代更新簇中心，以最小化“簇内平方和（SSE）”为目标，使同一簇内的样本尽可能紧密地聚集在一起，实现数据的自动聚类划分。

目标函数定义：

$$SSE = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

其中 μ_i 为第 i 个簇的均值向量，该函数衡量了簇内样本到中心的距离平方和。

02 标准算法步骤（迭代闭环）

01 初始化

随机从数据集中选择 k 个样本，将其作为初始的簇中心，为后续分配提供基准。

02 分配样本

计算每个样本到所有簇中心的距离（如欧氏距离），将样本分配给距离最近的簇中心所在的簇。

03 更新中心

对每个簇内的所有样本计算均值向量，将该均值作为对应簇的新中心，完成一次参数迭代。

04 收敛终止

重复执行“分配-更新”过程，直到簇中心的位置不再发生显著变化，算法收敛并输出最终聚类结果。

K-means: 关键点与优缺点



01. K值的科学选择

无法直接通过公式推导，需借助肘部法则（Elbow Method）或轮廓系数（Silhouette Coefficient）量化评估，以确定数据集中最优的聚类数量 k 。



02. 初始中心的敏感性优化

随机初始质心易陷入局部最优，实际中常用 k-means++ 算法优化，通过使初始中心尽可能分散，有效提升聚类结果的稳定性与准确性。



03. 对异常值的鲁棒性不足

算法基于距离度量计算簇中心，极端异常值会显著拉偏质心位置，导致聚类结果偏离真实分布，数据预处理阶段需重点进行异常值的检测与剔除。



核心优势：高效且易落地

原理直观简单，计算复杂度为 $O(nkt)$ （ n 为样本数， k 为簇数， t 为迭代次数），收敛速度快。作为经典的无监督学习算法，其实现门槛低，非常适合处理大规模数据集，是工业界最常用的聚类方法之一。



主要局限：场景适配性受限

需预先人工指定簇数 k ，缺乏自适应性；对初始质心和异常值均较为敏感。同时，算法仅能有效发现球形、凸状的簇结构，对非凸、不规则形状的簇识别效果较差，难以适配数据分布复杂的场景。

HC算法：核心思想与凝聚法步骤



核心思想：构建层次化簇树

层次聚类（Hierarchical Clustering）以“自底向上”的聚合策略为核心，从数据集中每个样本独立构成一个初始簇开始，通过迭代计算簇间相似度，逐步合并距离最近的簇。这一过程持续至所有样本归为单一簇，最终形成具有嵌套关系的树状层级结构（树状图/Dendrogram），揭示数据内在的层级关联。

关键特征：无需预先指定聚类数量，结果呈现数据的完整层级谱系。

1

初始化簇空间

将数据集中的每个样本点视为一个独立的初始簇，此时数据集被划分为 n 个簇（ n 为样本总数），构成聚类的最底层基础。

2

计算簇间相似度矩阵

依据选定的距离度量（如欧氏距离、曼哈顿距离），计算当前所有簇两两之间的相似度/距离，构建完整的簇间距离矩阵。

3

迭代合并最近邻簇

查找距离矩阵中相似度最高（距离最小）的两个簇并将其合并为新簇；更新距离矩阵，重复此过程直至所有样本聚合为单一簇。

4

截取树状图确定最终聚类

在生成的树状图（Dendrogram）的指定高度绘制水平切割线，切割线与树状图的交点数量即为最终聚类簇的数量，实现聚类结果的提取。

HC算法：簇间距离度量方法

在层次聚类（Hierarchical Clustering）中，核心问题是如何量化两个簇之间的相似性，即定义簇间“距离”。不同的度量方式会直接影响聚类树的结构与最终的簇形态。

01 单链接 (Single Linkage)

核心定义：将两个簇之间的距离定义为簇中所有样本对里，距离最小的那一对样本的距离，即取“最近邻”距离。

算法特点：易受链式效应影响，容易形成细长的簇结构，对异常值的耐抗性较差。

02 全链接 (Complete Linkage)

核心定义：将两个簇之间的距离定义为簇中所有样本对里，距离最大的那一对样本的距离，即取“最远邻”距离。

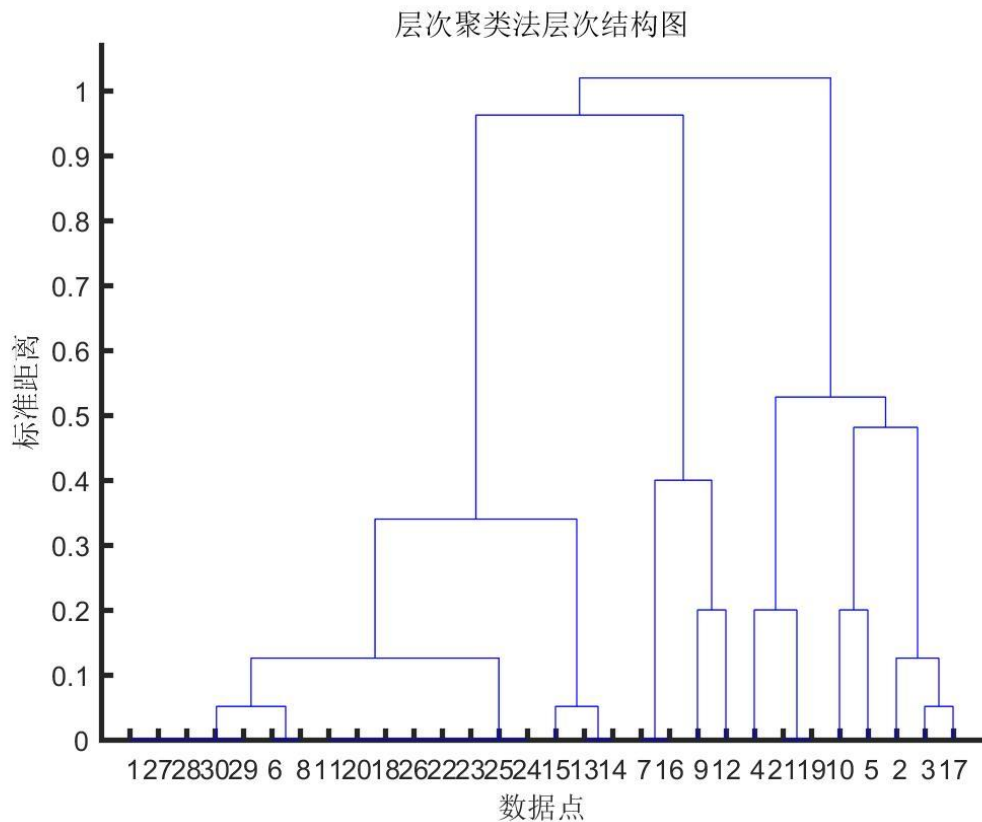
算法特点：对异常值高度敏感，倾向于生成紧凑、球形的簇，能有效避免链式效应，但易被极端值主导结果。

03 平均链接 (Average Linkage)

核心定义：计算两个簇中所有样本对之间的距离，取其算术平均值作为簇间距离，反映簇的整体相似度。

算法特点：鲁棒性较好，平衡了单链接和全链接的极端情况，既不会过度拉伸簇，也不易被单个异常值干扰。

HC聚类结果图解读（以5类为例）



树状图通过层级合并展示样本间的亲疏关系，垂直方向的高度差直观反映了簇与簇之间的差异程度。

01. 树状图核心结构解析



样本轴 (X)

横轴代表参与聚类的所有样本对象，每个样本初始为独立簇，随距离升高逐步合并。



距离轴 (Y)

纵轴表示簇合并时的相似度距离，距离数值越高，说明被合并的两个簇之间的差异越大。



阈值水平线

人为设定的分割线，其与树状图垂直线的交点数量，即为最终划分出的簇的总数。

02. 聚成5类的关键操作步骤

STEP 1 确定阈值

在树状图纵轴选取合适高度（约0.4-0.5）绘制水平线。

STEP 2 统计交点

观察水平线与树状图垂直枝干的交点数量，此即为簇数。

STEP 3 结果确认

微调阈值线高度，确保恰好切分出5个独立、无交叉的簇。

《白话机器学习》 K-means章读书笔记

01. 核心思想：直观的“找重心分组”

算法逻辑如同日常分组：先随机放置K个聚类中心，将样本分配到距离最近的组，再重新计算各组的重心并移动，重复迭代直至重心位置稳定不再变化。

02. 核心问题：随机初始化的局部最优陷阱

随机选取初始中心可能导致算法收敛到局部最优解，而非全局最优。可采用k-means++算法优化：让初始中心尽可能互相远离，显著提升聚类效果的稳定性。

03. 模型选择：用“肘部法则”确定最优K值

以K值为横轴，误差平方和(SSE)为纵轴作图。当K值增加到某个点后，SSE下降速率会骤减，形成“肘部”，该点即为兼顾聚类效果与复杂度的最优K值。

04. 场景优化：针对不同问题的改进变体

Mini-batch K-means通过随机抽取小批量样本更新中心，大幅提升大规模数据的计算效率；K-medoids则使用实际样本点作为中心，替代均值计算，对数据中的异常值具备更强的鲁棒性。

本章小结：K-means是无监督聚类中最经典的算法，以高效、易解释为核心优势。在实践中，需结合肘部法则科学选择K值，并利用k-means++等改进策略规避局部最优，结合场景需求选择Mini-batch或K-medoids变体以获得更好效果。

总结：K-means vs. 层次聚类

对比维度	K-means 算法	层次聚类 (Hierarchical Clustering)
聚类结果形态	生成单一、扁平的簇划分结果，所有数据点仅归属于一个最终簇，无任何层级嵌套关系。	构建完整的树状（谱系）层次结构，可通过截断树状图在任意层级灵活选择簇的数量。
计算效率与数据规模	时间复杂度低 ($O(nkt)$)，迭代收敛速度快，能够高效处理十万甚至百万级的大规模数据集。	时间复杂度高 ($O(n^3)$)，计算成本昂贵，仅适用于中小规模数据集，难以应对海量数据场景。
参数与鲁棒性：K-means需人工预先指定K值，且对异常值/离群点极度敏感；层次聚类无需预设簇数，对异常值的鲁棒性相对更强（具体依赖簇间距离计算方法）。		核心适用场景：K-means适用于凸形/球形簇、大规模数据的快速聚类；层次聚类更适合小数据集的探索性分析、非凸形状簇挖掘，以及需要展示数据层级关系的研究场景。